Apriori and Association Rules - Parallel versions with Multiprocessing and Joblib

Matteo Lotti
7154910
matteo.lotti3@edu.unifi.it

Abstract

This project focuses on the parallel implementation of the Apriori algorithm and the generation of association rules used in Market Basket Analysis. These algorithms are exploited for the implementation of cross selling strategies, inventory management, promotion planning and layout optimization. Generally speaking, after a data preprocessing phase, Apriori is used to generate frequent itemsets, useful to identify sets of items bought frequently together; then, association rules evaluates how strongly those items are related and how buying a certain item or group of items influences the customer to buy something else. The aim of the project is to implement parallel versions of Apriori and Association Rule generation, evaluate their benefits with respect to the sequential version and compare them to identify the best solution for parallelization.

1. Introduction

The Apriori algorithm is a fundamental method in data mining, primarily designed for the discovery of frequent itemsets in transactional datasets. These frequent itemsets form the foundation for association rule generation, a subsequent step that identifies meaningful correlations between items. Association rules are widely applied in domains such as market basket analysis, recommendation systems, and customer behavior modeling, where uncovering relationships among items provides insights that could lead to effective business decisions. While Apriori is conceptually simple and effective, its iterative candidate generation and repeated scans of large datasets make it computationally and memory-wise expensive. This bottleneck becomes especially evident when mining massive real-world datasets, where sequential execution often struggles to meet practical time requirements. With the purpose of overcoming these limitations, parallel implementations have been developed, distributing both candidate counting and rule generation across multiple processes. Frameworks such as Python's multiprocessing library and Joblib enable such parallelization, each with distinct trade-offs in performance, memory efficiency, and overhead. Comparing sequential and parallel executions of Apriori, together with the rule generation phase, offers valuable insights into how parallelism can accelerate frequent pattern mining.

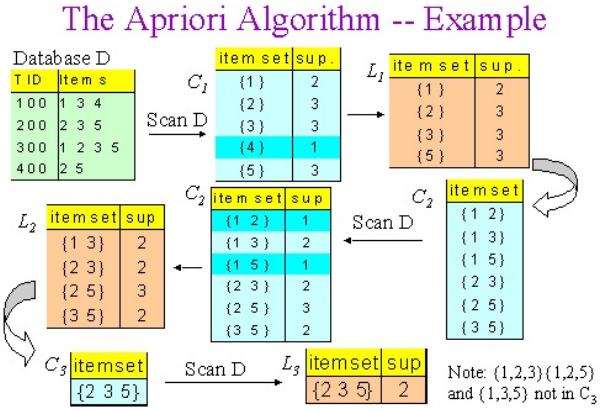

Figure 1. Apriori Algorithm Example

2. Implementation of the Apriori algorithm and Association Rule Generation

The workflow of the Apriori algorithm with association rule generation can be divided into few main phases, each consisting of distinct steps:

1. Transaction Preprocessing
2. Frequent Itemset mining with Apriori
   - Candidate generation
   - Support counting
   - Pruning based on minimum support threshold
3. Generation of Association Rules
   - Rule construction
   - Confidence Evaluation

Transaction Preprocessing The process begins with preparing the dataset. Each transaction is represented as a set of items, assuming that the algorithm can operate on a set-based structure. This step may involve cleaning the data, removing irrelevant elements, and grouping items by transaction identifiers.

Frequent itemset mining with Apriori The Apriori algorithm starts by generating candidate itemsets of size one. Their support, defined as the proportion of transactions in which an itemset appears, is computed. Only those itemsets that meet or exceed the minimum support threshold are considered frequent.

The procedure then iterates: larger candidate itemsets are formed by combining smaller frequent itemsets. For each candidate, support is counted again, and non-frequent itemsets are pruned. This process repeats, with the size of itemsets increasing at each iteration, until no further frequent itemsets can be identified.

Generation of Association Rules Once the set of frequent itemsets is complete, the algorithm proceeds to generate association rules. For each frequent itemset, possible rules are constructed by dividing it into an antecedent and a consequent.

Each rule could be evaluated using multiple metrics; in this case rules are weighted using confidence, which measures how often the consequent appears in transactions that contain the antecedent. Only rules with confidence above the specified threshold are retained. The outcome of this phase is a collection of association rules that highlight meaningful and significant relationships among items.

In the following subsections Python sequential and parallel code will be presented , as well as the implementation choices made for parallelization.

2.1. Sequential Version

The sequential implementation follows a straightforward structure that mirrors the conceptual workflow of Apriori and association rule generation. Transactions are first loaded from the input dataset, either in a "long" format, for very large and heavy datasets, grouped by transaction identifiers or in a standard matrix-like form, and converted into sets of items (*load_transactions* and *load_transactions_from_long*). The core Apriori procedure (*apriori*) begins by generating candidate itemsets of size one and computing their support with *count_support*. The function *filter_frequent* then keep only those candidates that satisfy the minimum support threshold. This process is repeated iteratively, with new candidates created by joining frequent itemsets of the previous iteration, until no further frequent itemsets can be discovered. All frequent itemsets and their support values are stored for later use. Once frequent itemsets are identified, the algorithm moves to the second phase, where the function *generate_association_rules* constructs potential rules by partitioning itemsets into antecedent–consequent pairs. For each rule, confidence is computed and compared against the user-defined minimum confidence threshold; rules meeting this requirement are preserved.

```python
def load_transactions(path):
    # loads transaction from dataset

def load_transactions_from_long(path):
    # loads transaction from dataset (long format)

def count_support(candidates, transactions):
    support = defaultdict(int)
    for transaction in transactions:
        for candidate in candidates:
            if candidate.issubset(transaction):
                support[candidate] += 1
    return support

def filter_frequent(support_count, minsup, n_transactions):
    return {
        itemset: count / n_transactions
        for itemset, count in support_count.items()
        if (count / n_transactions) >= minsup
    }

def apriori(transactions, minsup):
    n_transactions = len(transactions)
    items = sorted({item for transaction in transactions for item in transaction})
    L = []
    support_data = {}

    candidates = [frozenset([item]) for item in items]
    k = 1
    while candidates:
        support_count = count_support(candidates, transactions)
        Lk = filter_frequent(support_count, minsup, n_transactions)
        if not Lk:
            break
        L.append(Lk)
        support_data.update(Lk)
        prev_frequent = list(Lk.keys())
```

```
        candidates = [i.union(j) for i in
            prev_frequent for j in
            prev_frequent if len(i.union(j))
             == k + 1]
        candidates = list(set(candidates))
        k += 1
    return L, support_data

def generate_association_rules(
    frequent_itemsets, support_data,
    min_conf):
    rules = []
    for k_itemsets in frequent_itemsets
        [1:]:
        for itemset in k_itemsets.keys():
            for i in range(1, len(itemset))
                :
                for antecedent in itertools
                    .combinations(itemset, i
                    ):
                    antecedent = frozenset(
                        antecedent)
                    consequent = itemset -
                        antecedent
                    if consequent:
                        conf = support_data
                            [itemset] /
                            support_data[
                            antecedent]
                        if conf >= min_conf
                            :
                            rules.append((
                                antecedent,
                                consequent,
                                support_data
                                [itemset],
                                conf))
    return rules
```

Listing 1. Sequential code

2.2. Parallel version

To explore various implementation opportunities for parallelization, three parallel version were implemented:

1. Using the *multiprocessing* library
2. Using the *Joblib* library
3. Using the *Joblib* library coupled with serialization strategies for memory mapping of transaction and support counts

2.2.1 multiprocessing

The multiprocessing version extends the sequential Apriori implementation by introducing parallel computation in the two most computationally intensive phases: support counting and association rule generation. The key design choice was to exploit Python's *multiprocessing.Pool* to distribute the workload across multiple processes, thereby "bypassing" the Global Interpreter Lock (GIL) and allowing true parallel execution on multiple CPU cores.

For support counting, the set of candidate itemsets is divided into chunks, each assigned to a worker process. A global variable is initialized with the full transaction list (*init_worker*), avoiding repeated data transfer to worker processes. Each worker (*support_worker*) is then responsible for scanning the entire transaction database but only for its assigned candidate subset, returning partial support counts. These partial results are merged in the main process into a single support dictionary. This chunking strategy balances workload distribution and avoids memory overhead from copying large transaction structures between processes.

A similar approach is adopted for association rule generation. Once frequent itemsets have been identified, all itemsets larger than size one are collected and split into chunks. Each chunk is processed independently by a worker function (*rules_worker*), which generates antecedent–consequent pairs and evaluates their confidence against the minimum confidence threshold. The results from all workers are combined into the final list of rules.

An important aspect of the implementation is the flexibility of workload distribution. Both in support counting and rule generation, the chunk size is either automatically computed based on the number of processes and items, or explicitly defined by the user. This allows adjusting parallelization, trading off between reduced communication overhead and better load balancing.

Overall, this multiprocessing design achieves parallelism by focusing on task partitioning of the most expensive loops in Apriori, while preserving the sequential structure for tasks like candidate generation and filtering. This ensures correctness while potentially reducing runtime on large datasets.

```
    _transactions = None

def init_worker(transactions):
    global _transactions
    _transactions = transactions

def load_transactions(path):
    # ...same as sequential...

def load_transactions_from_long(path):
    # ...same as sequential...

def support_worker(candidates_chunk):
    support = defaultdict(int)
    for transaction in _transactions:
        for candidate in candidates_chunk:
```

```
            if candidate.issubset(
                transaction):
                support[candidate] += 1
    return support

def count_support_parallel(candidates,
    transactions, n_processes, chunk_size):
    chunks = [candidates[i:i + chunk_size]
        for i in range(0, len(candidates),
        chunk_size)]
    with multiprocessing.Pool(processes=
        n_processes, initializer=init_worker
        , initargs=(transactions,),) as pool
        :
        results = pool.map(support_worker,
            chunks)

    merged = defaultdict(int)
    for partial in results:
        for itemset, count in partial.items
            ():
            merged[itemset] += count
    return merged

def filter_frequent(support_count, minsup,
    n_transactions):
    return {
        itemset: count / n_transactions
        for itemset, count in support_count
            .items()
        if (count / n_transactions) >=
            minsup
    }

def apriori_parallel(transactions, minsup,
    n_processes=4, chunk_size=None):
    # ...similar to sequential...
    while candidates:
        if chunk_size is None:
            chunk_size_eff = len(candidates
                ) // n_processes + 1
        else:
            chunk_size_eff = chunk_size

        support_count =
            count_support_parallel(
            candidates, transactions,
            n_processes, chunk_size_eff)

    # ...similar to sequential...

def rules_worker(args):
    itemsets_chunk, support_data, min_conf
        = args
    rules = []
    for itemset in itemsets_chunk:
        for i in range(1, len(itemset)):
            for antecedent in itertools.
                combinations(itemset, i):
                antecedent = frozenset(
                    antecedent)
                consequent = itemset -
                    antecedent
                if consequent:
                    conf = support_data[
                        itemset] /
                        support_data[
                        antecedent]
                    if conf >= min_conf:
                        rules.append((
                            antecedent,
                            consequent,
                            support_data[
                            itemset], conf))
    return rules


def generate_association_rules_parallel(
    frequent_itemsets, support_data,
    min_conf, n_processes=4, chunk_size=None
    ):
    all_itemsets = [itemset for k_itemsets
        in frequent_itemsets[1:] for itemset
         in k_itemsets.keys()]
    if not all_itemsets:
        return []

    if chunk_size is None:
        chunk_size_eff = len(all_itemsets)
            // n_processes + 1
    else:
        chunk_size_eff = chunk_size
    chunks = [all_itemsets[i:i +
        chunk_size_eff] for i in range(0,
        len(all_itemsets), chunk_size_eff)]

    with multiprocessing.Pool(processes=
        n_processes) as pool:
        results = pool.map(rules_worker, [(
            chunk, support_data, min_conf)
            for chunk in chunks])

    rules = [rule for partial in results
        for rule in partial]
    return rules
```

Listing 2. multiprocessing code

2.2.2 Joblib

The joblib version of Apriori follows the same general workflow as the multiprocessing approach, but parallelization is handled through the joblib library, which provides a higher-level and more flexible abstraction for distributing tasks. The motivation for this choice is that joblib simplifies workload distribution by automatically managing task

scheduling, process spawning, and result collection, while still enabling execution across multiple cores.

For support counting, the set of candidate itemsets is divided into chunks, each assigned to a parallel job via *joblib.Parallel* and *delayed*. Each worker (*support_worker_chunk*) processes its subset of candidates against the full transaction database, computing partial support counts. Once all workers complete their tasks, the main process merges the partial dictionaries into a single support structure. This design preserves the correctness of Apriori.

Candidate generation and filtering remain sequential, but the costly support counting phase benefits from distributed execution. As in the multiprocessing version, the chunk size can either be specified by the user or automatically computed from the number of jobs and candidates.

For association rule generation, a similar principle is applied: itemsets larger than size one can be divided into chunks, with each chunk processed independently by the function *rules_single*. Each job computes antecedent–consequent pairs and evaluates their confidence, returning only those rules that satisfy the minimum confidence threshold.

Overall, the joblib implementation represents a cleaner and more maintainable parallelization strategy compared to "plain" multiprocessing. By abstracting away process management details, it allows for more concise code and easier experimentation.

```
def load_transactions(path):
    # ...same as sequential...

def load_transactions_from_long(path):
    # ...same as sequential...


def support_worker_chunk(candidates_chunk, transactions):
    #...similar to multiprocessing...

def count_support_joblib(candidates, transactions, n_jobs, chunk_size=None):
    chunks = [candidates[i:i + chunk_size] for i in range(0, len(candidates), chunk_size)]
    results = Parallel(n_jobs=n_jobs, backend="multiprocessing")(
        delayed(support_worker_chunk)(chunk, transactions) for chunk in chunks
    )
    merged = defaultdict(int)
    for partial in results:
        for itemset, count in partial.items():
            merged[itemset] += count
    return merged


def filter_frequent(support_count, minsup, n_transactions):
    #...same as multiprocessing...

def apriori_joblib(transactions, minsup, n_jobs=4, chunk_size=None):
    #...similar to multiprocessing...
        support_count = count_support_joblib(candidates, transactions, n_jobs, chunk_size_eff)
    #...similar to multiprocessing...

def rules_single(support_data, min_conf, itemset_chunk=None):
    # ...same as multiprocessing rules_worker...



def generate_association_rules_joblib(frequent_itemsets, support_data, min_conf, n_jobs=4, chunk_size=None):
    #...similiar to multiprocessing...

    results = Parallel(n_jobs=n_jobs, backend="multiprocessing")(
        delayed(rules_single)(support_data, min_conf, chunk) for chunk in chunks
    )
    rules = [rule for partial in results for rule in partial]
    return rules
```

Listing 3. Joblib code

2.2.3 Joblib with Memory Mapping

The joblib coupled with serialization strategies implementation extends the joblib version by addressing the issue of memory sharing across processes. In multiprocessing environments, large datasets must often be copied by each worker process, leading to redundant memory usage and possible termination of jobs on memory-constrained systems. To mitigate this, the code leverages memory mapping through joblib's *dump* and *load* functions. Instead of passing large transaction or support structures directly, these are stored on disk and accessed in read-only mode by all worker processes. This allows multiple processes to efficiently share the same data without duplicating it in memory.

The workflow begins with the *save_memmap* function, which serialize the transaction dataset to a memory-mapped file. During support counting, workers no longer receive the full transactions list as an in-memory object; instead, they

reload it locally from the shared file using the *load_memmap* function. The *support_worker_chunk* function accesses transactions through memory mapping, iterating over them and computing partial support counts for its assigned candidate subset. After parallel execution through Parallel, the partial dictionaries are merged by the main process to produce the global support counts.

The Apriori loop (*apriori_joblib_memmap*) preserves the same iterative candidate generation and filtering logic as in the previous versions, but now all support counting tasks operate over memory-mapped files. This design minimizes memory overhead and enables scaling to datasets that would not fit in RAM when duplicated across processes.

Association rule generation follows the same principle. The *rules_worker* function reloads the support data from a memory mapping file inside each worker, instead of requiring a large dictionary to be copied to every process. Rules are then generated in parallel with Parallel, with each worker computing confidence values for its assigned chunk of itemsets and returning only those above the threshold.

Overall, the momory-mapping-based joblib implementation is designed for scalability with very large datasets. It trades some I/O overhead for much lower memory usage, ensuring that parallel jobs can be executed without exhausting system resources. This makes it particularly suitable when handling lots of transactions or very dense itemsets, where memory consumption can otherwise become a bottleneck. At the same time, it preserves the modular structure of the joblib version, with minimal changes to the workflow.

```
def load_transactions(path):
    #...same as sequential...

def load_transactions_from_long(path):
    #...same as sequential...

def save_memmap(transactions, filename="
    transactions.pkl"):
    dump(transactions, filename)
    return filename

def load_memmap(filename="transactions.pkl"
    ):
    return load(filename, mmap_mode='r')

def support_worker_chunk(candidates_chunk,
    transactions_file):
    transactions = load_memmap(
        transactions_file)
    support = defaultdict(int)
    for transaction in transactions:
        for candidate in candidates_chunk:
            if candidate.issubset(
                transaction):
                support[candidate] += 1
    return support


def count_support_joblib_chunks(candidates,
     transactions_file, n_jobs_eff,
    chunk_size):
    chunks = [candidates[i:i + chunk_size]
        for i in range(0, len(candidates),
        chunk_size)]
    results = Parallel(n_jobs=n_jobs_eff,
        backend="multiprocessing")(
        delayed(support_worker_chunk)(chunk
            , transactions_file) for chunk
            in chunks
    )
    merged = defaultdict(int)
    for partial in results:
        for itemset, count in partial.items
            ():
            merged[itemset] += count
    return merged


def filter_frequent(support_count, minsup,
    n_transactions):
    #...same as multiprocessing...


def apriori_joblib_memmap(transactions_file
    , minsup, n_jobs=4, chunk_size=None):
    transactions = load_memmap(
        transactions_file)
    n_transactions = len(transactions)

    #...similar to multiprocessing...
        support_count =
            count_support_joblib_chunks(
            candidates, transactions_file,
            n_jobs, chunk_size_eff)
    #...similar to multiprocessing...


def rules_single(support_data, min_conf,
    itemset_chunk):
    # ...same as multiprocessing
        rules_worker...

def rules_worker(chunk, support_memmap_file
    , min_conf):
    support_data = load_memmap(
        support_memmap_file)
    return rules_single(support_data,
        min_conf, chunk)


def generate_association_rules_joblib(
    frequent_itemsets, support_memmap_file,
    min_conf, n_jobs=4, chunk_size=None):
    #...similiar to multiprocessing...
```

```
    results = Parallel(n_jobs=n_jobs,
        backend="multiprocessing")(
        delayed(rules_worker)(chunk,
            support_memmap_file, min_conf)
            for chunk in chunks
    )
    rules = [rule for partial in results
        for rule in partial]
    return rules
```

Listing 4. Joblib with Memory Mapping code

3. Experiments and Results

Testbed features Experiments have been run on an AMD Ryzen 7 5700G CPU, which features 8 core and 16 threads and operates at 3.8 GHz by default, but can boost up to 4.6 GHz, depending on the workload. The underlying operating system for the testst was Linux.

3.1. Experiments setup

The Apriori implementations were evaluated on two datasets: a grocery transactions dataset with 9,835 records, and a larger retail market basket dataset from an anonymous Belgian store containing 100,000 transactions. Experiments were conducted using three different minimum support thresholds (0.01, 0.02, and 0.05) to analyze the sensitivity of performance with respect to itemset density. For the parallel benchmarks, the number of processes (or jobs) was varied as powers of two, ranging from 2 to 64, in order to assess scalability across different levels of paralellization. Regarding chunk size, two configurations were tested: a minimal value (*chunk_size = 1*) and the adaptive value used by default in the implementations (computed as the maximum effective chunk size). The experiments revealed that *chunk_size = 1* introduced a significant overhead due to the excessive number of very small tasks being scheduled, which led to degraded performance. For this reason, results are reported only for the default chunk size configuration, which provides a better balance between task granularity and parallel execution efficiency.

3.2. Results

The experimental results are presented in terms of execution times and speedups for both datasets and all implementations. As shown in Tables 1, 2, 3 and 4, the rule generation phase consistently required significantly less time compared to the Apriori phase, with execution times lower than a second. Interestingly, the sequential version achieved lower rule generation times than the parallel ones, most likely because of the overhead introduced by multiprocessing and memory mapping; in these cases execution times are dominated by the process managment. For this reason, speedup plots (figures 2, 3, 4, 5, 6, 7) report only Apriori execution times. Regarding the Apriori phase, three implementations were compared: Joblib (JB), Joblib with memory mapping (JBMM), and raw multiprocessing (MP). For both the Groceries and Retail datasets, parallel versions generally improved performance up to a certain number of processes, with MP showing the highest speedups across almost all thresholds. JB and JBMM displayed smaller but still consistent gains, while sequential execution remained the slowest baseline. The trend highlights that performance increased steadily when going from 2 to 16 processes, reaching peak speedups in this range. However, for larger configurations (32 and 64 processes), execution times grew up, and the speedup curves declined, indicating that parallel efficiency dropped when exceeding a certain number of workers, especially with the respect to the number of cores and threads of the testbed's CPU (paragraph 3). Finally, the results also show how increasing the minimum support threshold reduced execution times overall, since fewer frequent itemsets were generated and evaluated, but also reduced speedups, as support threshold has a stronger influence on the sequential part of the code.

4. Conclusions

The obtained results highlight how the performance of parallel Apriori is strongly influenced by both the dataset characteristics and the experimental parameters. The Groceries dataset, being smaller in size and containing fewer transactions, showed limited opportunities for parallelism, with speedups remaining modest and overhead becoming dominant as the number of processes increased. In contrast, the Retail dataset, which is significantly larger, allowed the parallel versions to better exploit multiprocessing, especially at lower support thresholds where more candidate itemsets are generated. This explains why MP consistently achieved the highest speedups: its lower abstraction overhead allowed it to scale more effectively when the workload was sufficiently large. On the other hand, JB and JBMM, while easier to implement and manage, introduced additional overhead due to job scheduling and abstractions, limiting their gains. In particular, JBMM performed worse than plain Joblib with Groceries dataset, as the datasets used in the experiments was small enough to fit entirely in memory, making memory mapping unnecessary; in this case, the additional I/O and serialization overhead hid any potential benefit. On the other hand, they obtained similar performances with Retail dataset, as it is bigger than the other one and let the JBMM version show its potential usefulness. Moreover, when testing the Retail dataset, JB and especially JBMM generally performed slightly better with low number of processes (2 or 4), but MP achieved the best speedups when the number of processes increase.

Regarding the impact of the minimum support threshold,

lower values led to higher execution times but also greater scalability, since more candidate itemsets were generated and the workload could be better distributed across processes. Conversely, higher thresholds drastically reduced the number of candidates, shrinking the parallel section of the algorithm and leaving the sequential part dominant. As a result, execution times decreased, but speedup gains also dropped, since the sequential code benefited proportionally more from higher thresholds than the parallel part. Finally, the performance drop beyond 8 or 16 processes can be explained by the limited number of physical cores and by inter-process communication overhead: as the number of workers surpassed the available hardware resources, context switching, synchronization, and memory contention dominated the benefits of parallelization, leading to reduced efficiency.

5. Appendix - Tables and Plots

Support Threshold	Apriori Time (s)	Rules Time (s)
1%	44.491750	0.000140
2%	37.750442	0.000051
5%	36.868816	0.000015

Table 1. Retail Sequential Times

Support Threshold	Apriori Time (s)	Rules Time (s)
1%	2.029105	0.000225
2%	0.628118	0.000051
5%	0.141054	0.000004

Table 2. Groceries Sequential Times

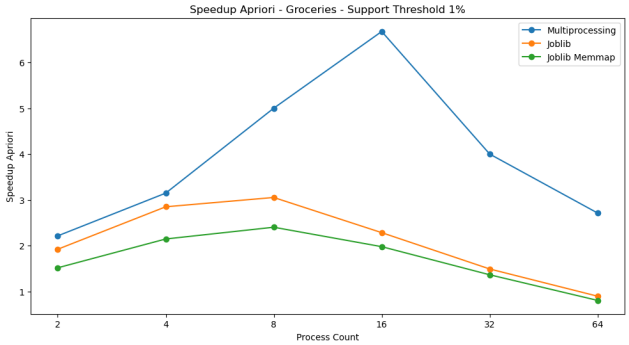

Figure 2. Groceries Speedup - Support Threshold 1%

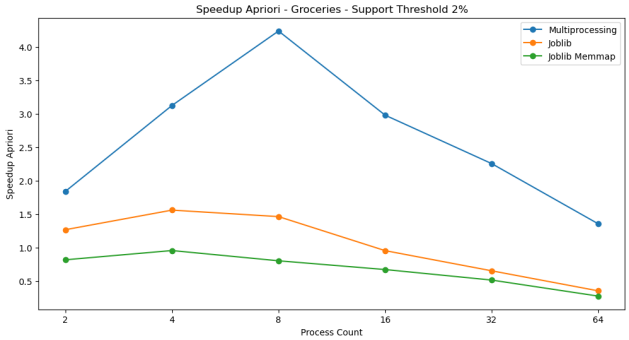

Figure 3. Groceries Speedup - Support Threshold 2%

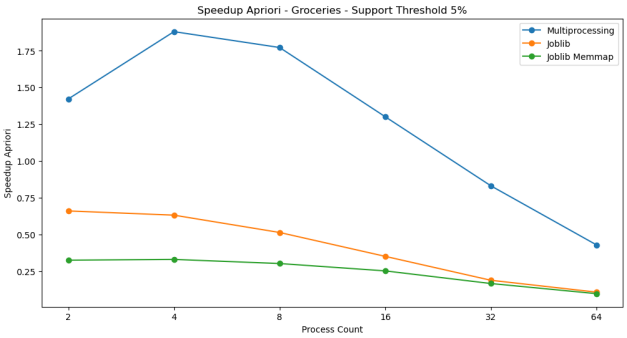

Figure 4. Groceries Speedup - Support Threshold 5%

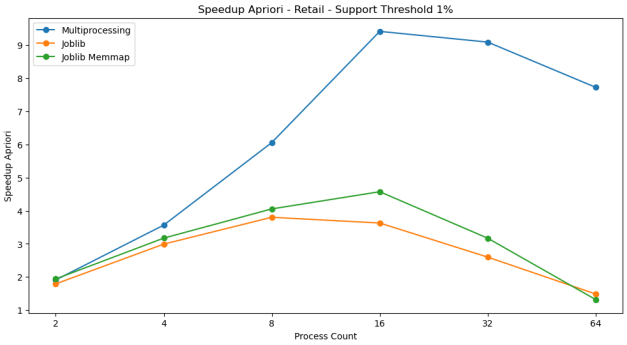

Figure 5. Retail Speedup - Support Threshold 1%

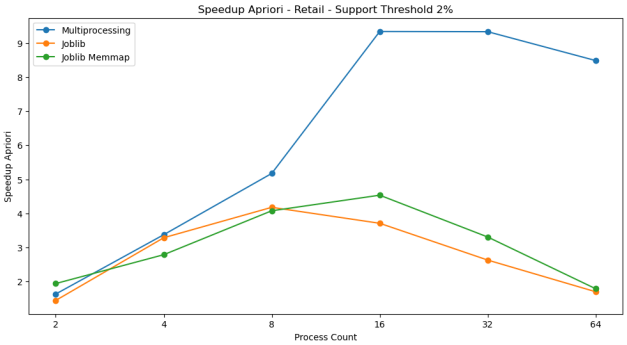

Figure 6. Retail Speedup - Support Threshold 2%

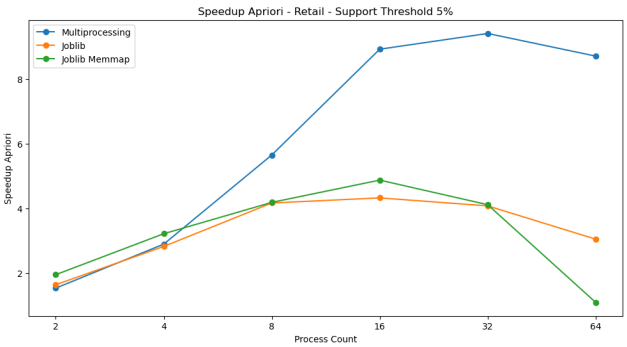

Figure 7. Retail Speedup - Support Threshold 5%

		Apriori Time (s)			Rules Generation Time (s)		
Support Threshold	Version⟶ Process Amount↓	JB	JBMM	MP	JB	JBMM	MP
1%	2	24.817878	22.907988	23.270734	0.065068	0.066606	0.009624
	4	14.869697	14.022639	12.477325	0.069953	0.071423	0.012656
	8	11.703591	10.975263	7.348446	0.079030	0.080276	0.021829
	16	12.261387	9.729478	4.728487	0.093909	0.095969	0.032766
	32	17.112471	14.032408	4.896793	0.146654	0.138510	0.058519
	64	29.927544	33.725732	5.761934	0.279845	0.305048	0.124386
2%	2	26.093082	19.438800	23.095046	0.067012	0.070063	0.008632
	4	11.488494	13.537719	11.186068	0.071806	0.077117	0.011354
	8	9.035430	9.259353	7.299148	0.080431	0.086876	0.017428
	16	10.176033	8.325753	4.043462	0.097708	0.106380	0.032424
	32	14.350989	11.414917	4.046107	0.130137	0.151805	0.055400
	64	22.195012	21.096293	4.450189	0.216789	0.276993	0.115269
5%	2	22.561778	18.946742	24.120646	0.065857	0.069835	0.008098
	4	13.073680	11.470355	12.751565	0.070935	0.074892	0.011771
	8	8.849106	8.809200	6.522688	0.077227	0.084576	0.017018
	16	8.525824	7.562476	4.128032	0.100220	0.103853	0.028136
	32	9.043623	8.962422	3.914617	0.126365	0.162973	0.072465
	64	12.104875	33.965583	4.229391	0.213962	0.253162	0.115824

Table 3. Retail Parallel Times

		Apriori time (s)			Rules Generation time (s)		
Support Threshold	Version⟶ Process Amount↓	JB	JBMM	MP	JB	JBMM	MP
1%	2	1.054708	1.335758	0.915731	0.041293	0.074795	0.009293
	4	0.711819	0.944684	0.644365	0.039339	0.080051	0.011120
	8	0.664441	0.843908	0.405668	0.046596	0.086161	0.016877
	16	0.887658	1.024800	0.303917	0.062096	0.106773	0.028615
	32	1.359170	1.485238	0.507190	0.093499	0.157910	0.052841
	64	2.258554	2.515681	0.747989	0.170716	0.258169	0.127387
2%	2	0.495133	0.767347	0.341080	0.036758	0.071493	0.007103
	4	0.402278	0.656272	0.200948	0.039562	0.076544	0.010442
	8	0.428889	0.781443	0.148157	0.045236	0.089119	0.016824
	16	0.657258	0.933462	0.210690	0.056624	0.111953	0.029371
	32	0.960303	1.218490	0.278336	0.089689	0.144670	0.046523
	64	1.773115	2.279011	0.463215	0.155809	0.272148	0.097672
5%	2	0.213864	0.435082	0.099166	0.037367	0.071402	0.006857
	4	0.223577	0.428071	0.075004	0.039278	0.076579	0.010015
	8	0.274986	0.467949	0.079576	0.043547	0.086122	0.015526
	16	0.402694	0.561236	0.108412	0.059316	0.102083	0.025237
	32	0.752411	0.853348	0.169803	0.089784	0.140072	0.048806
	64	1.324341	1.465697	0.329371	0.140338	0.227989	0.094533

Table 4. Groceries Sequential Times